

头文件

```
#include<ext/pb_ds/assoc_container.hpp> //必写
#include<ext/pb_ds/tree_policy.hpp> //用tree
#include<ext/pb_ds/hash_policy.hpp> //用hash
#include<ext/pb_ds/trie_policy.hpp> //用trie
#include<ext/pb_ds/priority_queue.hpp> //用priority_queue

using namespace __gnu_pbds;
```

平衡树

定义

```
tree<
    Key,                // 键类型
    Mapped,             // 值类型（若为集合则用 null_type）
    Compare,            // 比较函数（如 less<int>）
    Tag,                // 树类型标签（rb_tree_tag）
    Node_Update         // 节点更新策略（tree_order_statistics_node_update）
> tr;
//example
tree<pair<int, int>, null_type, less<pair<int, int>>, rb_tree_tag,
tree_order_statistics_node_update> tr;
```

操作

```
tr.insert(make_pair(x, y)); //插入;
tr.erase(make_pair(x, y)); //删除;
tr.order_of_key(make_pair(x, y)); //base 0 求排名 （即小于该元素的个数）
tr.find_by_order(k); //base 0 查询排名为 k 的元素（即第 k+1 小的元素）返回迭代器
tr.join(b); //将b并入tr, 前提是两棵树类型一样且没有重复元素
tr.split(v, b); //分裂, key小于等于v的元素属于tr, 其余的属于b
tr.lower_bound(x); //返回第一个大于等于x的元素的迭代器
tr.upper_bound(x); //返回第一个大于x的元素的迭代器
```

元素不能重复,如果你想要重复的话,可以再开一维加上编号

```
//example
for (int i = 0; i < n; ++i) tr.insert(make_pair(a[i], i));

//然后这样找到小于一个元素x的个数
it = ps.lower_bound(make_pair(x, -1));
int ans = ps.order_of_key(*it);
```

priority_queue

定义

```
__gnu_pbds:: priority_queue<Key, // 键类型
                          Compare, // 比较函数（如 less<int>）
                          Tag // 实现方式
                          > Q;

//加上命名空间pbds，否则可能和<queue>的pq冲突

/*其中的TAG为类型，有以下几种：
pairing_heap_tag
thin_heap_tag
binomial_heap_tag
rc_binomial_heap_tag
binary_heap_tag
其中pairing_help_tag最快*/

/*使用合并的时候使用 pairing_heap_tag
做dijisktra的时候使用 thin_heap_tag*/

//example
__gnu_pbds:: priority_queue<int, less<int>, pairing_heap_tag> Q;
```

操作

1.基本操作

```
Q.push(x);
Q.pop();
Q.top();
Q.empty();
Q.size();
```

2.将另一个堆加入Q

```
Q.join(b);
```

3.删除修改

```
Q.modify(it,6);
```

```
Q.erase(it);
```

```
// 获取元素句柄 (Iterator/Handle)
```

```
// 插入元素时, push 方法会返回一个指向该元素的 句柄 (point_iterator), 后续可通过句柄操作该元素
```

```
auto it1 = Q.push(10); // 插入10, 获取句柄it1
```

```
auto it2 = Q.push(20); // 插入20, 获取句柄it2
```

```
auto it3 = Q.push(15); // 插入15, 获取句柄it3
```

```
Q.erase(it2); // 删除元素20, 堆中剩余{15, 10}
```

```
Q.modify(it, 50); //将it指向的元素修改为50, 并重新调整堆
```

4.分裂

```
Q.push(10); Q.push(20); Q.push(30); Q.push(15); // Q = {30, 20, 15, 10}
```

```
Q.split(15, Q2); // 以15为界分裂
```

```
// 结果:
```

```
// Q = {15, 10}      (所有 ≤15 的元素)
```

```
// Q2 = {30, 20}     (所有 >15 的元素)
```